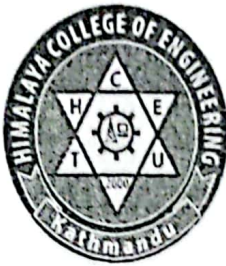
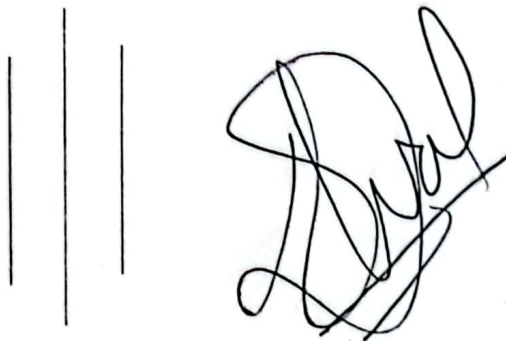




TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 5

Title:- Exploring Essential Linux Commands and
Thread management, with Gcc installation in Ubuntu

Submitted by:-

Submitted To:- Drish mali

Name:- Surajan Shrestha

Department Of CSIT

Roll No:- 38

Checked by:-

Date of submission:- 2081/05/20

TITLE :- Exploring Essential Linux Commands and Thread management, with Gcc Installation in ubuntu.

OBJECTIVE :-

- 1, To understand and practice basic linux Commands.
- 2, To learn the usage of directory manipulation Commands like ls, pwd, cd, mkdir, and rm.
- 3, To gain hands-on experience with file management Commands such as touch, nano and rm.
- 4, To explore the creation and editing of files in the terminal environment.
- 5, To comprehend the concepts of relative and absolute paths in a linux environment.

THEORY :-

→ In this lab, we explore a set of fundamental commands used in the linux operating system, as well as the concept of threads. Each Command serves a specific purpose, ranging from file and directory manipulation to displaying information and navigating the filesystem. Understanding these commands is crucial for efficient operation in a linux environment.

• Threads in operating systems:

→ A thread is the smallest unit of a process that can be scheduled and executed by the OS. In a multi-threaded environment, a process can contain multiple threads, each performing the different tasks simultaneously. Threads share the

some memory space within a process but have their own execution stack, program counter and local variables.

Threads are essential for improving the performance of applications, particularly those that involves tasks that can be executed concurrently. For instance, in a web server, different threads can handle individual client requests, enabling efficient and simultaneous processing. Threads can be managed through libraries like POSIX threads (pthreads) in Unix/Linux systems.

• Relative and Absolute paths:

→ A relative path refers to the location of a file or directory relative to the current working directory. An absolute path is the complete path from the root directory to the target file or directory. Understanding these concepts is essential for efficient navigation and file management in Linux.

• ls command:

→ The ls command is used to list the contents of a directory. It displays files, directories and their associated permissions, sizes and modification times. It is one of the most frequently used commands in Linux.

• pwd command:

→ The pwd command stands for "print working directory". It displays the full path of the current

working directory. This command is useful for confirming your location within the directory structure.

• cd commands:

→ The cd command is used to change the current directory. It allows you to navigate through the file system by moving between directories. The command can take relative or absolute paths as its argument.

• touch commands:

→ The touch command is used to create a new empty file or update the timestamps of existing files. It is a simple way to create files in the terminal.

• nano commands:

→ The nano command opens a text editor within the terminal. It is a simple and user-friendly editor that allows for creating and editing text files directly in the command line interface.

• gcc --version command:

→ The gcc --version command is used to check the installed version of the GNU compiler collection (GCC) on the system. GCC is a compiler system that supports various programming languages including C, C++, and Objective-C. This command is useful for verifying whether GCC is installed and determining its version, which is important for

compatibility with different programs & libraries.

- sudo apt update command:

→ The sudo apt update command is used to update the package lists on an Ubuntu system. This command fetches the latest versions of the package repositories, ensuring that the system is aware of the most recent software updates available. It is a necessary step before installing or upgrading software using the package manager.

OBSERVATION:

- gcc --version :

→

```
surajan-shrestha@Surajan-Shrestha: $  
surajan-shrestha@Surajan-Shrestha: $ gcc --version  
gcc (Ubuntu 13.2.0-23ubuntu4) 13.2.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

- use of ls & gcc hello.c -o test:

→

```
surajan-shrestha@Surajan-Shrestha: ~  
surajan-shrestha@Surajan-Shrestha: $ gcc hello.c -o test  
surajan-shrestha@Surajan-Shrestha: $ ls  
date.txt  error.txt  input.txt  program2.c  snap  test  
Desktop  file.txt   music      program3.c  surajan  Videos  
Documents hello.c    new.c      program.c   surajan.save  work  
Downloads helo.c     Pictures   Public      templates     xy.txt  
surajan-shrestha@Surajan-Shrestha: $ ./test  
Before  
After  
6977  
After  
0  
surajan-shrestha@Surajan-Shrestha: $
```

use of nano commands:



```
surajan-shrestha@Surajan-Shrestha: ~  
surajan-shrestha@Surajan-Shrestha: $ touch hello.c  
surajan-shrestha@Surajan-Shrestha: $ nano hello.c  
surajan-shrestha@Surajan-Shrestha: $ nano program.c  
surajan-shrestha@Surajan-Shrestha: $ nano program2.c  
surajan-shrestha@Surajan-Shrestha: $ nano program3.c  
surajan-shrestha@Surajan-Shrestha: $
```

```
GNU nano 7.2 hello.c  
#include<unistd.h>  
#include<stdio.h>  
void main()  
{  
    int id;  
    printf("Before \n");  
    id=fork();  
    printf("After \n");  
    printf("%d\n",id);  
}
```

```
GNU nano 7.2 program.c  
#include<unistd.h>  
#include<stdio.h>  
void main()  
{  
    int id;  
    printf("Before \n");  
    id=fork();  
    fork();  
    printf("After \n");  
    printf("%d\n",id);  
}
```

```
GNU nano 7.2 program2.c  
#include<unistd.h>  
#include<stdio.h>  
void main()  
{  
    int id;  
    printf("Before \n");  
    id=fork();  
    printf("After \n");  
    printf("Process ID : %d, \nParent ID : %d, \n",getpid(),getppid());  
    printf("Child ID : %d, \n",id);  
}
```

```
surajan-shrestha@Surajan-Shrestha:~  
GNU nano 7.2 program3.c *  
#include<unistd.h>  
#include<pthread.h>  
#include<stdio.h>  
void*routine()  
{  
printf("Test from Threads.\n");  
sleep(5);  
printf("Ending Thread.\n");  
}  
void main()  
{  
pthread_t t1,t2;  
pthread_create(&t1,NULL,&routine,NULL);  
pthread_create(&t2,NULL,&routine,NULL);  
pthread_join(t1,NULL);  
pthread_join(t2,NULL);  
}
```

• use of gcc filename.extension -o test & ./test

→

```
surajan-shrestha@Surajan-Shrestha:~  
surajan-shrestha@Surajan-Shrestha:~$ gcc program.c -o test  
surajan-shrestha@Surajan-Shrestha:~$ ./test  
Before  
After  
7028  
After  
7028  
After  
0  
After  
0  
surajan-shrestha@Surajan-Shrestha:~$
```

```
surajan-shrestha@Surajan-Shrestha:~$ gcc program2.c -o test  
surajan-shrestha@Surajan-Shrestha:~$ ./test  
Before  
After  
Process ID :7054,  
Parent ID : 3465,  
Child ID : 7055.  
After  
Process ID :7055,  
Parent ID : 7054,  
Child ID : 0.  
surajan-shrestha@Surajan-Shrestha:~$
```

```
surajan-shrestha@surajan-shrestha:~$ nano program3.c
surajan-shrestha@surajan-shrestha:~$ gcc program3.c -o test
surajan-shrestha@surajan-shrestha:~$ ./test
Test from Threads.
Test from Threads.
Ending Thread.
Ending Thread.
surajan-shrestha@surajan-shrestha:~$
```

DISCUSSION :

→ The exercises performed in this lab provide a foundational understanding of basic Linux commands essential for any users working in a Linux environment. The commands covered include directory & file management, navigation through the file system, and text editing. Mastery of these commands is crucial for more advanced tasks in Linux such as scripting & system administration.

Additionally, the use of text editors like nano within terminal environment introduces users to the concept of in-terminal text editing, a skill necessary for managing configuration files and writing the scripts in Linux.

CONCLUSION :

→ The lab successfully demonstrated the use of basic Linux commands, providing hands on experience with essential tasks such as directory navigation, file management & text editing. These skills form the building block for more advanced operations in Linux, providing the way for deeper exploration of the OS. Regular practice of these command will lead to increased efficiency & confidence when working in Linux environment.